

Tecnológico de Costa Rica

Introducción a Python

Ing. Laura Coto Sarmiento. MSc.



EMPRESAS QUE USAN PYTHON

YAHOO!

Google

 Instagram

facebook

 Spotify

 reddit



IBM

 Dropbox

Quora

 UBER

yelp

 NYSE

 INDUSTRIAL
LIGHT & MAGIC

 You
Tube

NETFLIX

 BATTLEFIELD
2

Pinterest

CÓMO INICIARTE EN PYTHON DE FORMA EFECTIVA



1. Aprender Lógica de Programación, tipos de datos y manejo de variables.

2. Realizar operaciones lógicas y aritméticas con diferentes tipos de datos.



3. Estudiar la utilización de ciclos y sentencias condicionales.

4. Aprender Programación Funcional y Programación Orientada a Objetos.



5. Manejo de Errores, Módulos, archivos y Datos Masivos.

5 RAZONES PARA APRENDER PYTHON



1. RECOMENDADO CÓMO PRIMER LENGUAJE DE PROGRAMACIÓN



Gracias a su sintaxis sencilla y limpia, es un **lenguaje rápido de aprender**.



2. MULTIPARADIGMA



Se puede **programar orientando a objetos** o utilizando programación **funcional o imperativa**.



3. DESARROLLO RÁPIDO



Puedes crear aplicaciones **en menos líneas de código** que muchos otros lenguajes.



5. SEGURIDAD INFORMÁTICA



Es uno de los lenguajes **más utilizados** y **adoptado** por la **industria de la seguridad informática**



4. MACHINE LEARNING

Python es el lenguaje más usado en **modelos de Machine Learning**.



En EDteam tenemos un curso de Python desde cero:

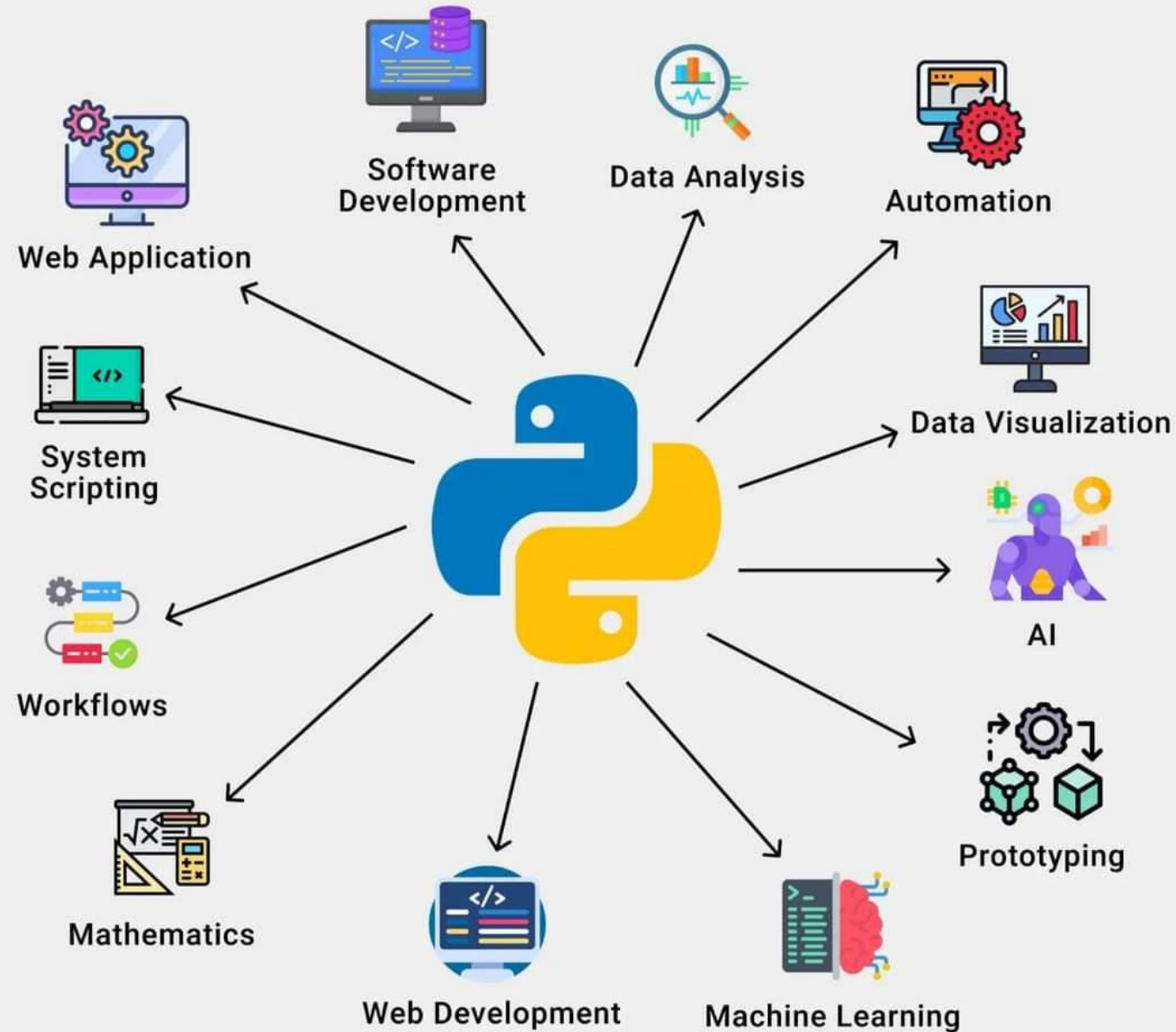


ed.team/cursos/python



WHAT PYTHON CAN DO

@coders.learning



Agenda



Entorno de desarrollo
Expresiones y sentencias
Variables
Tipos de Datos
Operadores
Comentarios
Identación
Estructuras de control
Errores

Entorno de desarrollo

El interprete de Python puede ser invocado para interpretar un archivo con `python ejemplo.py` o para abrir el entorno de comandos por consola con `python`

Python IDLE: Los programas de entorno de desarrollo integrado (IDE en inglés) proveen un ambiente para facilitar el desarrollo de programas, entre ellas pueden estar:

- Editar
- Compilar (en caso necesario)
- Depurar
- Ejecutar

Entorno de desarrollo

Python tiene un IDE nativo
(viene instalado con Python):

IDLE Python GUI

Entorno de desarrollo

Otros entornos integrados de desarrollo

(IDE: Integrated Development Environment):

- Visual Studio Code
- Geany
- NbPython
- Ninja
- BOA Constructor
- Eclipse + pydev
- Komodo
- Etc.

Los 5 mejores Entornos de Desarrollo Integrado (IDE) para Python



Desarrollado por JetBrains es un IDE multiplataforma considerado como uno de los mejores. Tiene dos versiones una gratuita y otra de pago.

Disponible en:



Fue creado en Java y puede instalarse como un complemento en Eclipse. Proporciona un análisis de código, depuración, refactorización y más.

Disponible en:



Es un IDE de código abierto y multiplataforma que viene por defecto con Python. Fue escrito en Python y es excelente para desarrolladores de nivel principiante que quieran iniciar en la programación con Python.

Disponible en:



SPYDER

The Scientific Python Development Environment

Scientific Python Development es utilizado principalmente por científicos de datos ya que se integra fácilmente con múltiples paquetes científicos. Tiene características avanzadas como: edición, depuración y exploración de datos.

Disponible en:



THE INTELLIGENT DEVELOPMENT ENVIRONMENT FOR PYTHON

Creado por Wingware es un IDE multiplataforma rápido, estable y extremadamente ligero. Cuenta con tres versiones: una versión de prueba gratuita y dos más de pago.

Disponible en:



IDE'S For PYTHON Programming



Python Basic Ide (Easy 😊)

(Easy To Use and Beginner Friendly)



Pycharm (complicated 😲)

(For Intermediate and Expert Programmer)



Sublime Text3 (Easy 😍)

(More Than A Text Editor. Easy To Use and small size.
Need Some configuration to Run Python)



Visual Code (Little Complicated 😬)

(An Intermediate Level ide. Best Recommended
Ide And Efficient to code in Different
programming languages)

Entorno de desarrollo

IDE de Python

Ambiente gráfico nativo de Python

Formas de uso

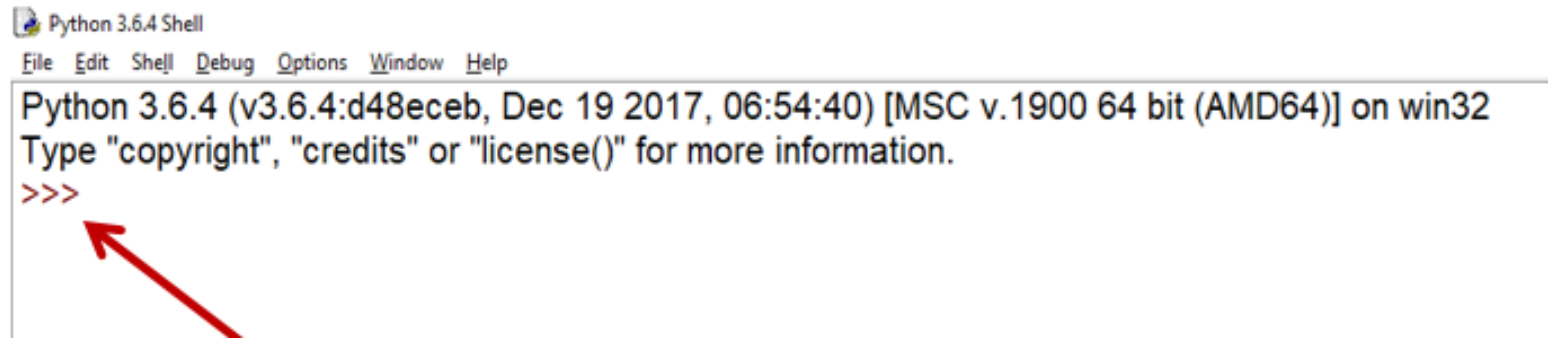
- Modo comando (shell)
- Modo programa (script)

Entorno de desarrollo

Modo comando (**shell** de Python)

En este modo hay un prompt (señal) “>>>” que indica que el intérprete esta esperando recibir instrucciones

Escribimos una instrucción y cuando la finalizamos damos la tecla <Enter> para que el intérprete la analice, si esta correcta la pasa a lenguaje de máquina y la ejecuta inmediatamente



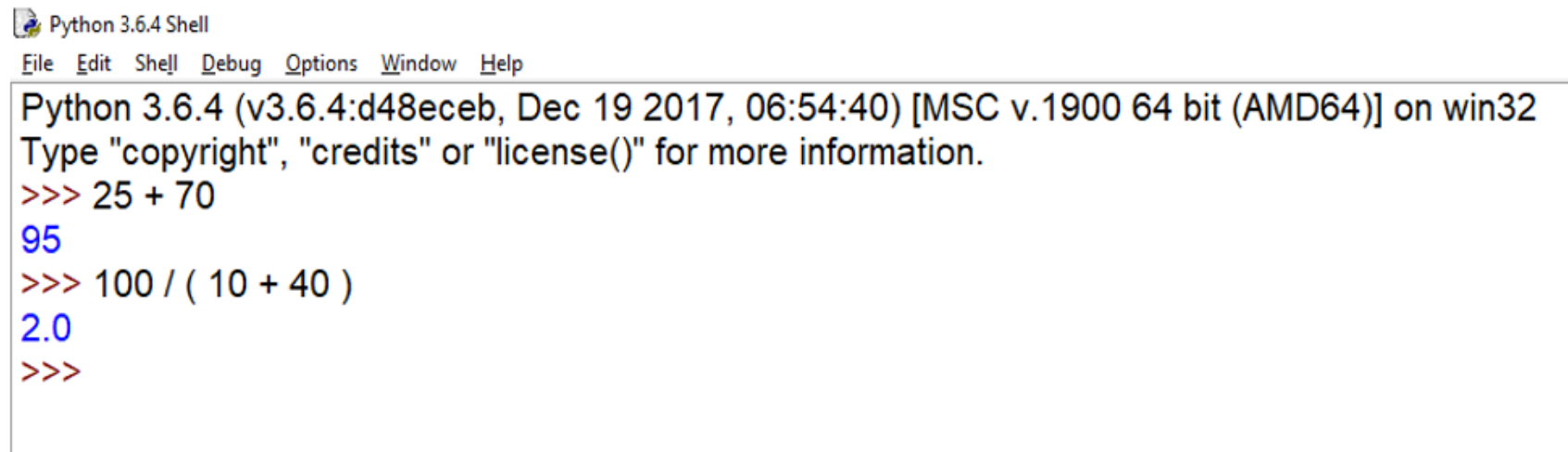
```
Python 3.6.4 Shell
File Edit Shell Debug Options Window Help
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
```

Entorno de desarrollo

Este modo es muy práctico ya que permite probar en forma interactiva las instrucciones

- Cuando estamos programando y tenemos dudas sobre alguna instrucción podemos usar este modo para hacer pruebas
- Luego de asegurarnos que la instrucción esta correcta la incorporamos al programa

En este modo Python se puede usar también como calculadora

A screenshot of a Python 3.6.4 Shell window. The window has a title bar that says "Python 3.6.4 Shell" and a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main area shows the Python version and build information: "Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32". It also displays the prompt "Type 'copyright', 'credits' or 'license()' for more information." followed by three interactive prompts and their results: ">>> 25 + 70" returns "95", ">>> 100 / (10 + 40)" returns "2.0", and ">>>" is shown on the next line.

```
Python 3.6.4 Shell
File Edit Shell Debug Options Window Help
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 25 + 70
95
>>> 100 / ( 10 + 40 )
2.0
>>>
```


Entorno de desarrollo

Modo programa (script)

Las instrucciones se escriben y guardan en un archivo: programa fuente

Posteriormente el programa guardado en ese archivo puede ser ejecutado por el intérprete

Extensión de los programas fuentes: **.py**

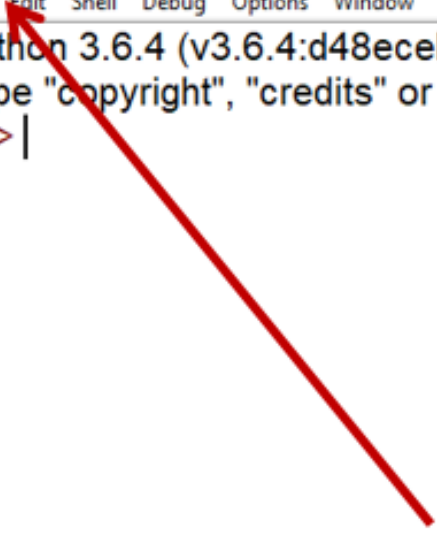
Python 3.6.4 Shell

File Edit Shell Debug Options Window Help

Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32

Type "copyright", "credits" or "license()" for more information.

>>> |



SELECCIONAR OPCIÓN: FILE

Python 3.6.4 Shell

File Edit Shell Debug Options Window Help

New File Ctrl+N

Open... Ctrl+O

Open Module... Alt+M

Recent Files ▶

Module Browser Alt+C

Path Browser

Save Ctrl+S

Save As... Ctrl+Shift+S

Save Copy As... Alt+Shift+S

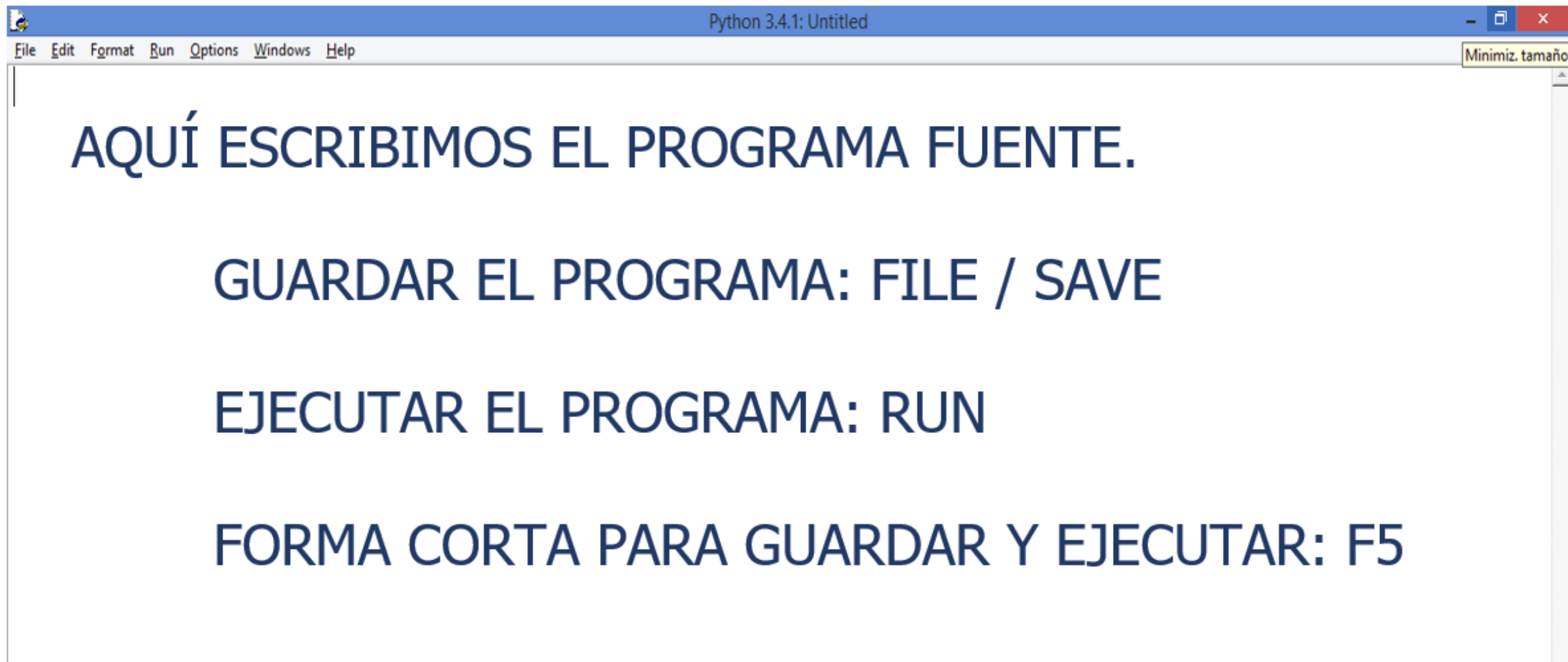
Print Window Ctrl+P

Close Alt+F4

Exit Ctrl+Q

d48eceb, Dec 19 2017, 06:54:40) [MSC v.1900 64 bit (AMD64)] on win32
dits" or "license()" for more information.

SELECCIONAR OPCIÓN: NEW FILE



Tipos de Datos

```
>>> type ("Hola mundo!")  
<class 'str'>
```

```
>>> type (3.2)  
<class 'float'>
```

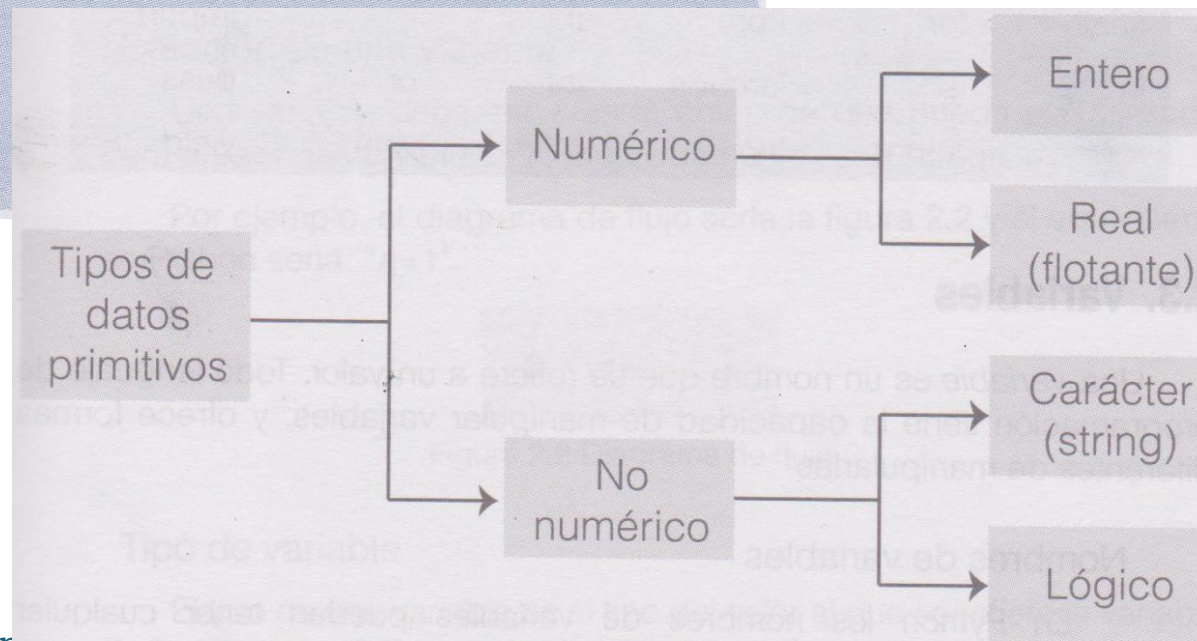
```
>>> pi = 3.14  
>>> type (pi)  
<class 'float'>
```

```
>>> n = 10  
>>> type (n)  
<class 'int'>
```

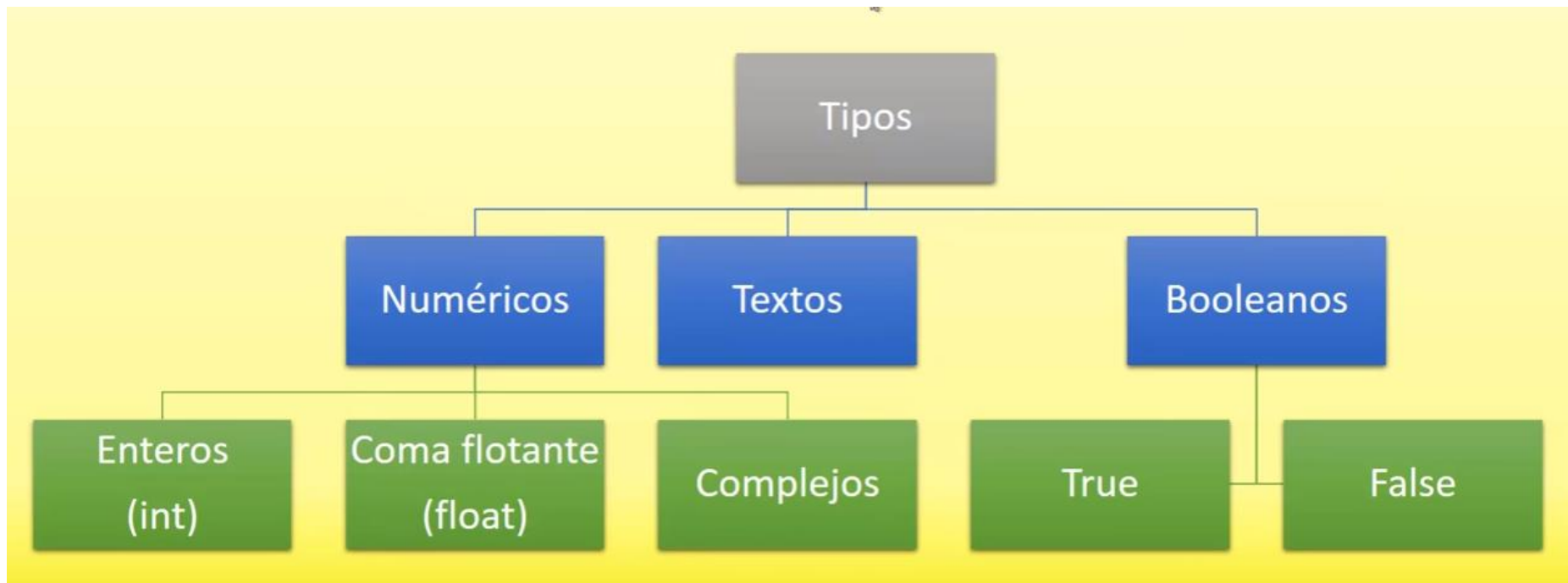
```
>>> type ("17")  
<class 'str'>
```

```
>>> type (17)  
<class 'int'>
```

```
>>> mensaje = "Hola"  
>>> type (mensaje)  
<class 'str'>
```



Videos recomendados...



<https://www.youtube.com/watch?v=u4l9PqhQCo8>

Tipos de datos

- Primitivos no numéricos:
- Secuenciales:

- Inmutables:

- Strings

```
>>> tira = "123"  
>>> tira[1] = "4"  
Traceback (most recent call last):
```

- tuplas

```
>>> tupla1 = (1, 2, 3)  
>>> tupla1  
(1, 2, 3)
```

- Mutables:

- Listas,

```
>>> lista1 = [2, 4, 6, 8]  
>>> lista2 = ['a', 'd', 'f']  
>>> lista_anidada = [[1, 2, 3], [4, 5, 6]]
```

- Arreglos

```
>>> arreglob = bytearray(b'xyz')  
>>> arreglob[0], arreglob[1], arreglob[2]  
(120, 121, 122)  
>>> arreglob[1] = 97
```

- Arreglos de byte mutables

- Conjuntos:

```
>>> deportes = {'basket', 'futbol', 'beisbol', 'futbol'}  
>>> deportes  
{'basket', 'beisbol', 'futbol'}
```

- Diccionarios:

```
>>> precios = {'LCD 50': 1200, 'Celular': 300,  
               'Refrigerador': 1500}  
>>> precios  
{'Refrigerador': 1500, 'LCD 50': 1200, 'Celular': 300}
```

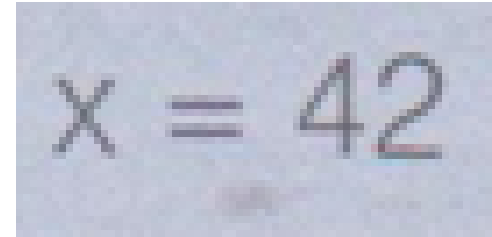
Palabras Reservadas

- No se pueden crear variables con su nombre.

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	not	or	pass
raise	return	try	while	with	yield

Nombres de Variables

- No puede ser el nombre de una palabra reservada.
- No debe iniciar con un número.
- Debe ser un nombre significativo.
- Se recomienda sea solo una palabra.
- Sin espacios ni caracteres especiales.

A photograph of a piece of paper with the handwritten text 'x = 42' in blue ink. The 'x' is written in a simple, slightly slanted font, followed by an equals sign and the number 42.

Función Print...



```
>>> print (1)  
1
```

```
>>> x = 2  
>>> print (x)  
2
```

```
>>> print (1,000,000)  
1 0 0
```

```
>>> p = print  
>>> p ("hola")  
hola  
>>> p("Hola" + "\n", "Estamos en Python")  
Hola  
Estamos en Python
```

```
>>> entrada = input ("¿Cuál es el nombre de tu universidad?")  
¿Cuál es el nombre de tu universidad? TEC  
>>> print (entrada)  
TEC
```

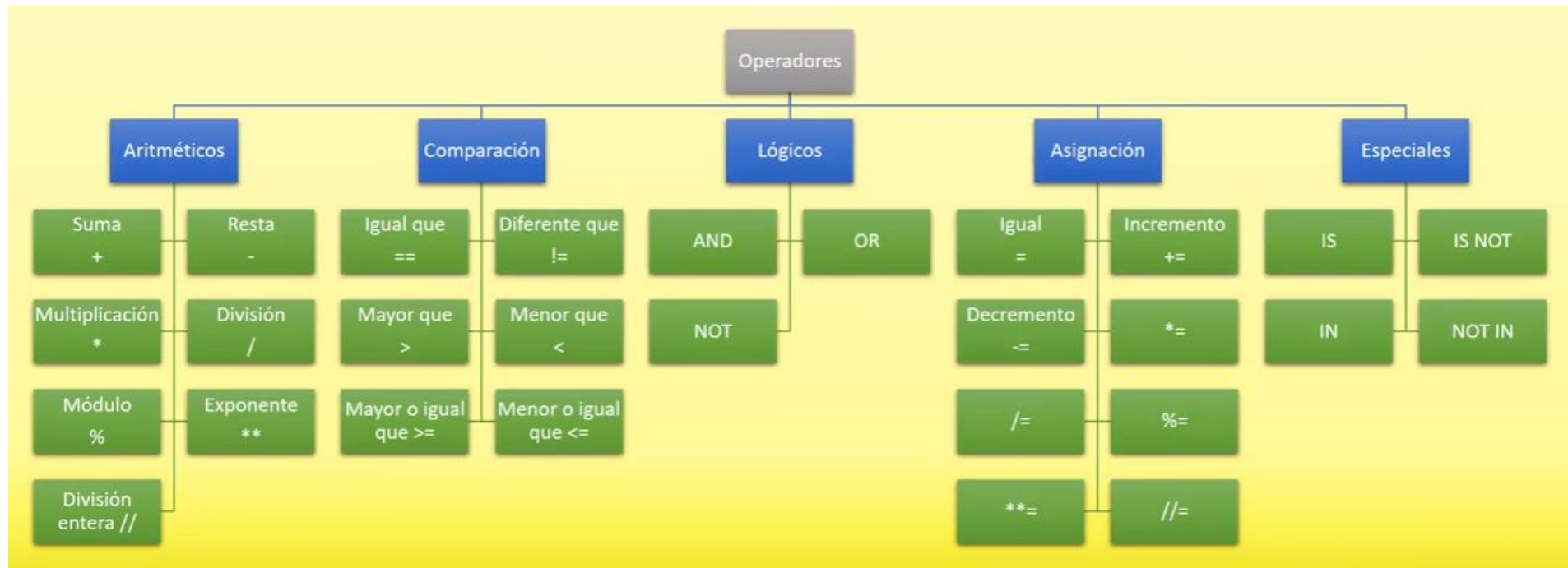
```
>>> a = int (input ("Digite un número entero: "))  
Digite un número entero: 34  
>>> a  
34
```



Asignación de variables

```
>>> x=42                                #asignación simple
>>> nombre, n = "Homero", 10           #asignación simultánea
>>> x = y = z = 10                      #asignación ligada
```

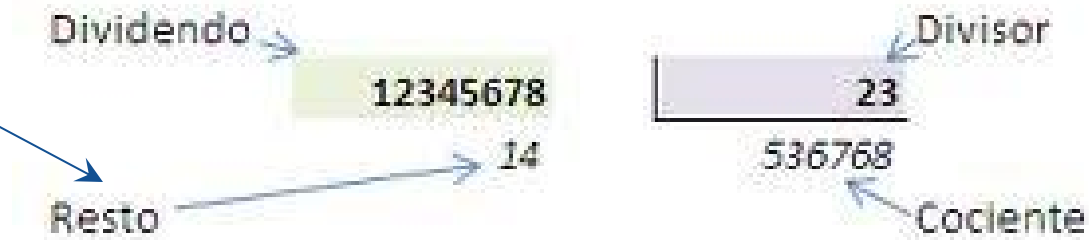
Uso de operadores...



<https://www.youtube.com/watch?v=u4I9PqhQCo8>

Operadores Matemáticos

Operación	Resultado	Operación	Resultado
$x+z$	Suma de x y z	$x-z$	Diferencia de x y z
$x*z$	Producto x y z	x/z	Cociente x y z
$x//z$	Cociente entero x y z	$x^{**}z$	X a la potencia de z
$x\%z$	Residuo x y z	$-x$	X negado



Las expresiones se evalúan usando el acrónimo
“**P**(parent)**E**(expon)**M**(multi)**D**(div)**A**(suma)**S**(resta)”

Calculadora

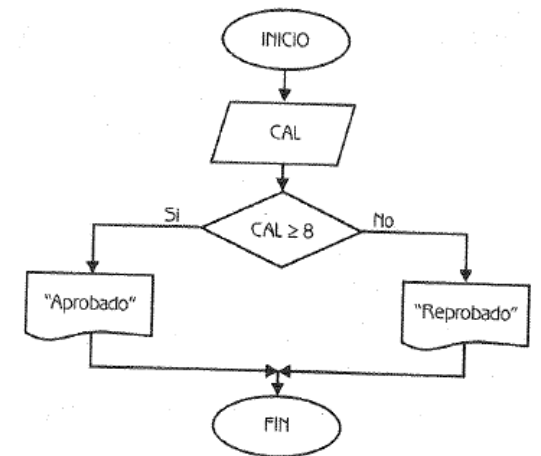
```
valor_a = 10      #una variable a la que le asigné el valor 10
valor_b = 5       #una variable a la que le asigné el valor 5
suma = valor_a + valor_b
resta = valor_a - valor_b
multi = valor_a * valor_b
divi = valor_a / valor_b
print ("La suma de a y b es: %d" % suma)
print ("La resta de a y b es: %d" % resta)
print ("La multiplicacion de a y b es: %d" % multi)
print ("La division de a y b es: %d" % divi)
```

Ejercicio

```
>>> minutos = 30
>>> hora = 12
>>> print("Número de minutos:", hora*60+minutos)
Número de minutos: 750
>>> porcentaje=(minutos*100)/60
>>> print(17+13, porcentaje)
30 50.0
```

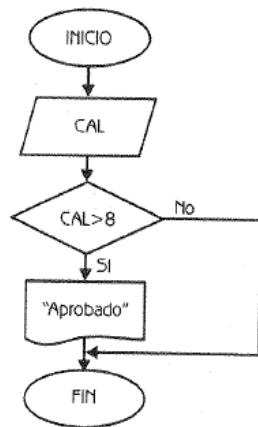

Operadores relacionales

Operador	Ejemplo	Significado
Igualdad	$X = y$	X es igual que y
Diferencia	$X \neq y$	X no es igual que y
Mayor	$X > y$	X es mayor que y
Menor	$X < y$	X es menor que y
Mayor o igual	$X \geq y$	X es mayor o igual que y
Menor o igual	$X \leq y$	X es menor o igual que y

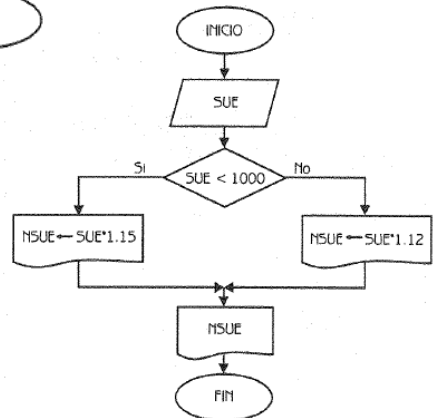


Condicionales y la indentación

- En Python la indentación representa la subanidación de código.
- Python lo asigna automáticamente.



```
cal=int(input("Indique la nota obtenida: "))
if cal>=80:
    print("Usted aprobó el curso")
```



```
x=int(input("Indique un valor para x: "))
y=int(input("Indique un valor para y: "))
if x==y:
    print("Los valores son iguales")
else:
    print("Los valores son diferentes")
```

Comentarios

- Sirven para posteriormente considerar aspectos de la solución del problema. Cosas que se pueden olvidar y son importantes recalcar. Símbolo #

```
#####  
#Script que suma dos valores #  
#####  
  
valor_a = 5 # Esto es un comentario en python  
valor_b = 3 # Los comentarios sirven para hacer tu codigo mas legible  
resultado = valor_a + valor_b # Aqui se le asigna el resultado de sumar a y b a la variable resultado  
print ("El resultado de sumar ", valor_a , " y " , valor_b , " es: " , resultado )|  
  
#####  
#Script que suma dos valores #  
#####  
  
valor_a = 5 # Esto es un comentario
```

**CUANDO ESCRIBÍ ESTE CÓDIGO,
SÓLO DIOS Y YO SABÍAMOS
CÓMO Y PARA QUÉ LO HICE**



AHORA, SÓLO DIOS LO SABE

Comentarios esperados

```
#Elaborado por: Laura Coto Sarmiento
#Fecha de Creación: 22/10/2017 8:00 a.m.
#Ultima modificación: 22/10/2017 11:00 a.m
#Versión de Python: 3.5.2

numero=int(input("Indique un número: "))
#Se analiza un número para ver si es un valor par o impar
if numero%2 == 0:
    print("El valor de la variable:",numero," es par.")
else:
    print("El valor de la variable:",numero," es impar.")
```


¿Qué está mal?

- Mejorar if y ver duplicidad en cada incremento

```
def analizarCadena(cadena):
    vocales=""
    consonantes=""
    espacios=""
    numeros=""
    simbolos=""
    i=0
    while i<len(cadena):
        if re.search(r"\d",cadena[i]):
            numeros=cadena[i]
            i+=1
            print(len (numeros))
        elif re.search(r"\s",cadena[i]):
            espacios=cadena[i]
            i+=1
            print(len (espacios))

        elif re.search(r"\W",cadena[i]):
            simbolos=cadena[i]
            i+=1
            print(len(simbolos))

        elif re.search("r\w",cadena[i]):
            vocales=cadena[i]
            consonantes=cadena[i]
            i+=1
            print(len(vocales))
            print(len(consonantes))

    return
```

tips

Es seguro | <https://www.youtube.com/watch?v=yppT6GPZMyo&t=4>

licaciones Save to Mendeley RELAJACIÓN MÚSICA MÚSICA EQUILIBRAR



Buscar

Python 3.6.0 Shell

File Edit Shell Debug Options Window Help

Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 07:18:10) [MSC v.1900 32 b

Type "copyright", "credits" or "license()" for more information.

```
>>> print ("Hola Alumnos")
```

```
Hola Alumnos
```

```
>>> print ("Hola alumnos"); print("Adios mundo cruel")
```

```
Hola alumnos
```

```
Adios mundo cruel
```

```
>>> #Aquí iría una comentario
```

```
>>> mi_nombre="mi nombre es Juan!"
```

```
>>> mi_nombre
```

```
'mi nombre es Juan!'
```

```
>>> mi_nombre="mi nombre es\
Juan"
```

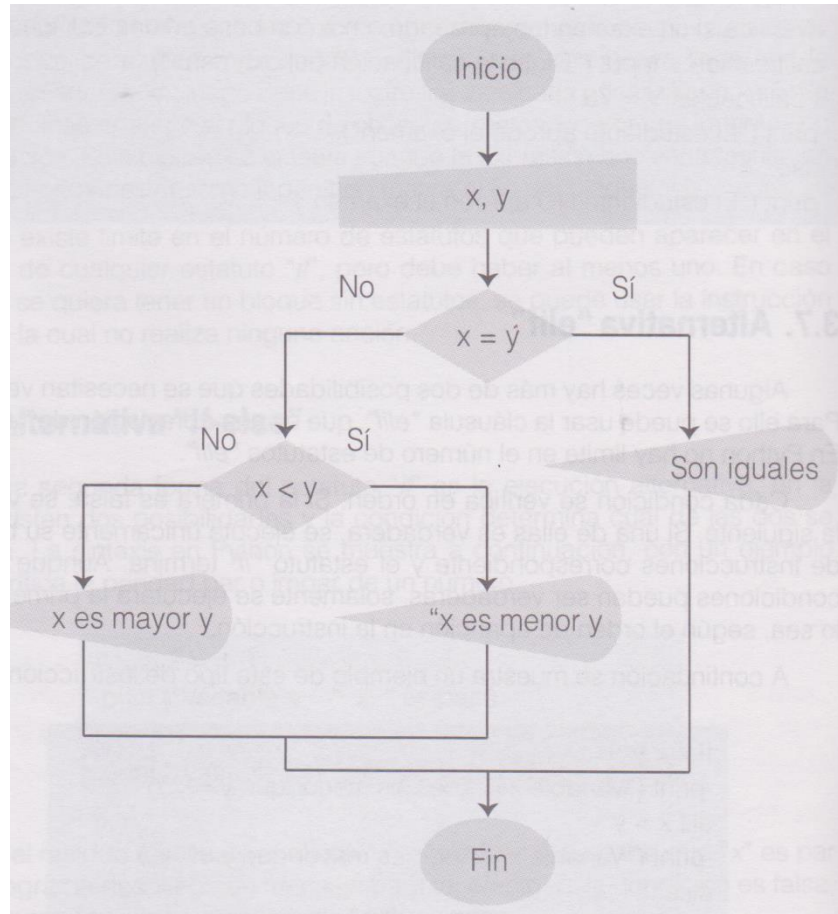
```
>>> |
```


Condicionales

```
#Programa 3.6.1.  
#Verifica si un examen fue aprobado o no, con base en una calificación  
calificacion = input ("Digite la calificación del examen: ")  
if calificacion >= 70 :  
    print ("El estudiante aprobó el examen")  
else :  
    print ("El estudiante NO aprobó el examen")
```

```
if x < y :  
    print ("Variable x = ", x, " es menor que y =", y)  
elif x > y :  
    print ("Variable x = ", x, " es mayor que y =", y)  
else :  
    print ("Variables x = ", x, " y =", y, "son iguales")
```


Anidación



```
If x == y :  
    print ("x=", x, "y=", y, " son iguales")  
else :  
    if x < y :  
        print ("x=", x, " es menor que y =", y)  
    else :  
        print ("x=", x, " es mayor que y =", y)
```

elif... selector...

- ```
x = int(input('Ingresa un entero, por favor: '))
if x < 0:
 x = 0
 print('Valor negativo, cambiado a cero')
elif x == 0:
 print('Es el valor Cero')
elif x == 1:
 print('Es un uno')
else:
 print('Es un número positivo mayor a 1')
```

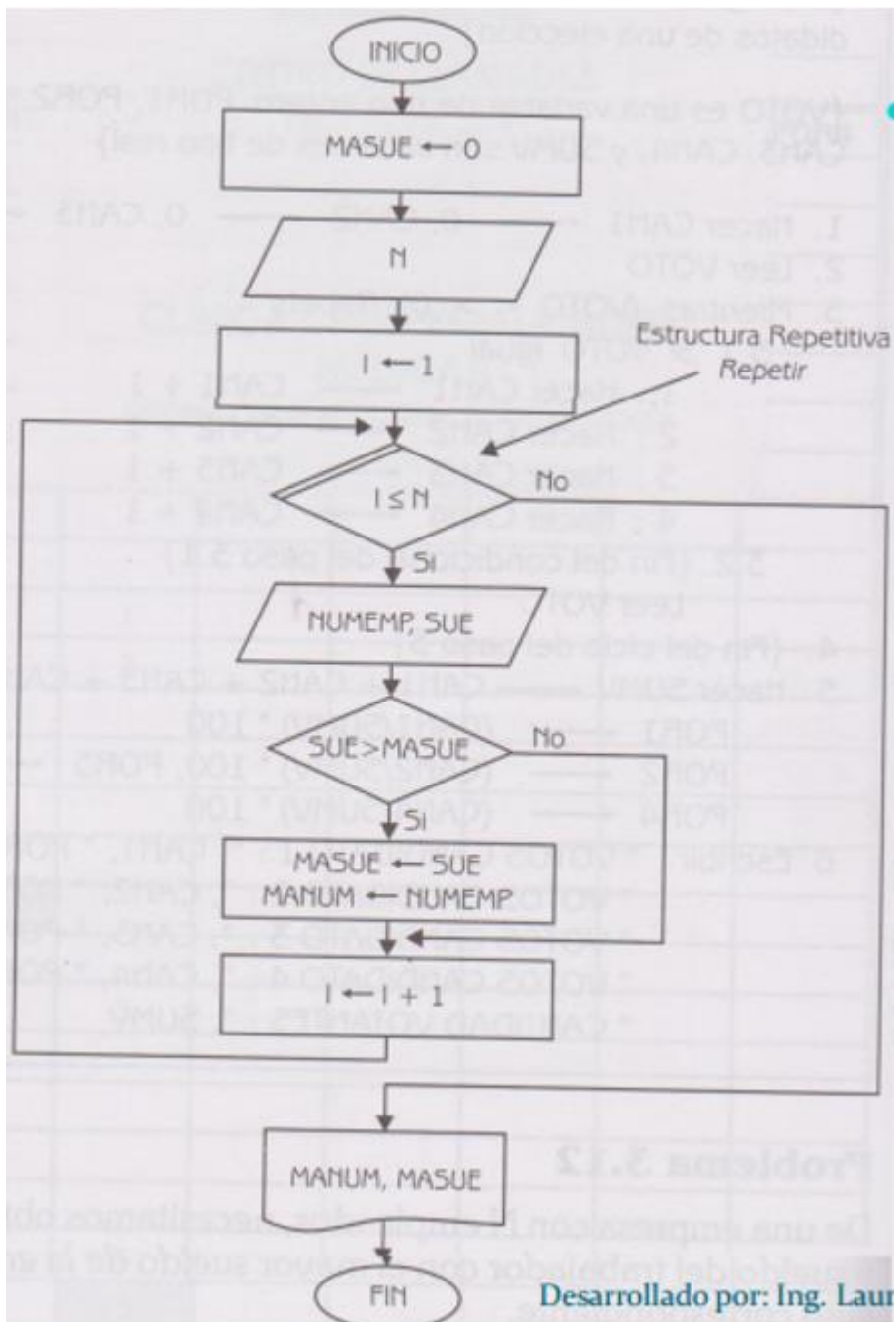


# Equivalencias

```
If 0 < x :
 if x < 10 :
 print ("x=", x, " está entre 0 y 10")

If 0 < x and x < 10 :
 print ("x=", x, " está entre 0 y 10")

If 0 < x < 10 :
 print ("x=", x, " está entre 0 y 10")
```



# While

- `valor = 0`  
`while valor < 10:`  
    `print ("contador", valor)`  
    `valor += 1`

- Salida es:

>>>

contador 0  
contador 1  
contador 2  
contador 3  
contador 4  
contador 5  
contador 6  
contador 7  
contador 8  
contador 9

# While

- ```
valor = 0
while valor < 10:
    print ("contador", valor)
    valor += 1
```

- Salida es:

```
>>>
contador 0
contador 1
contador 2
contador 3
contador 4
contador 5
contador 6
contador 7
contador 8
contador 9
```

for

- ```
for i in range(10):
 print("cuenta",i)
```

- Salida es:

```
>>>
cuenta 0
cuenta 1
cuenta 2
cuenta 3
cuenta 4
cuenta 5
cuenta 6
cuenta 7
cuenta 8
cuenta 9
```

# La función range()

Nos permite....

- >>> range(5, 10)  
[5, 6, 7, 8, 9]
- >>> range(0, 10, 3)  
[0, 3, 6, 9]

# Realicemos ejercicios...

- Usando el FOR con el range, incremente de 1 en 1 y obtenga el cuadrado de cada número, hasta el 11, de tal forma que la salida sea:

$$0*0=0$$

$$1*1=1$$

$$2*2=4$$

$$3*3=9$$

$$4*4=16$$

...

- Solicite un número y genere la potencia del número ingresado para los valores hasta el 10. Use un for o un While.

$$0**2=0$$

$$1**2=1$$

$$2**2=4$$

$$3**2=9$$

$$4**2=16$$

...

$$10**2=100$$



# While con break

- contador = 0  
while (contador < 5):  
 print ('El contador es :', contador)  
 contador = contador + 1  
 if (contador > 3):  
 break  
print ("Fin del programa")

>>>

El contador es : 0

El contador es : 1

El contador es : 2

El contador es : 3

Fin del programa

>>>

# Adivina...

```
1 numero = 23
2 detener = False
3 while not detener:
4 adivina = int(input("Ingrese un entero : "))
5 if adivina == numero:
6 print ("¡Felicitaciones, haz adivinado!.")
7 detener = True # Esto lo va a detener
8 elif adivina < numero:
9 print ("No, es un número mayor.")
10 else: # debes adivinar
11 print ('No, es un número menor.')
12 print('¡El ciclo while ha terminado!')
```

Ingrese un entero : 21  
No, es un número mayor.  
Ingrese un entero : 24  
No, es un número menor.  
Ingrese un entero : 23  
¡Felicitaciones, haz adivinado!.  
¡El ciclo while ha terminado!

# IF múltiple = Case

- ```
n = int(input("ingrese un número: "))  
if n == 0:  
    print("Su valor es 0.")  
elif n == 1 or n == 9 or n == 4:  
    print("n es un cuadrado perfecto")  
elif n == 2:  
    print("n es un número par")  
elif n == 3 or n == 5 or n == 7:  
    print("n es un número primo")
```

¿Qué está mal?

- contador = 0
while (contador < 5):
 print ('El contador es :', contador)
 contador = contador + 1
print ("Fin del programa")

La indentación...

Lo correcto

- contador = 0
while (contador < 5):
 print ('El contador es :', contador)
 contador = contador + 1
print ("Fin del programa")

Errores en un programa

Errores sintácticos: un programa solo puede ejecutarse si está correcto sintácticamente, si cumple las reglas de escritura definidas para el lenguaje

Errores semánticos: suceden en tiempo de ejecución y se asocian con la lógica utilizada para resolver el problema. Un programa puede ejecutarse correctamente pero retornar resultados incorrectos según el problema definido

Excepciones: errores en tiempo de ejecución esperados y controlados por el programador, que son comunicados al usuario, pero no abortan la ejecución del programa

Investigue sobre los estatutos “try” y “except”

```
#Error sintactico
variable = 56 + variable2 = 3;
```

Error sintáctico en una expresion

```
#Error semantico
arreglo = [1, 2, 3];
variable = arreglo[5];
```

Error semántico explícito

```
indice = 5;
arreglo = [1, 2, 3];
if(indice > len(arreglo))
#Excepcion
raise ValueError('Indice fuera
del rango permitido');
```

IDLE para Windows.

How people reacts differently to a
single word.

"Bug"



Tester



Developer



Manager

Python 3 • hoja de referencia: lo básico

PYTHON CHEAT SHEET FOR BEGINNERS

TIPOS DE DATOS

```
bool = True / False
int = 5
float = 29.60
str = "¡hola!"
```

OPERADORES NUMÉRICOS

```
+ suma
- resta
* multiplicación
/ división
** potencia
% módulo
// división entera
```

OPERADORES COMPARACIÓN

```
== igual
!= distinto
> mayor
< menor
>= mayor o igual
<= menor o igual
```

OPERADORES BOOLEANOS

```
and "y" lógico
or "o" lógico
not negación lógica
```

VARIABLES

CREACIÓN

```
radio=20
```

USO

```
radio**2
```



Programación
Desde Cero

STRINGS

CONCATENAR

```
"Hola " + "mundo"
```

OBTENER LONGITUD

```
len("chocolate")
```

CARÁCTER EN POSICIÓN 0

```
"Música"[0]
```

REBANADA

```
"margarita"[2:6]
```

ENTRADA / SALIDA DE DATOS

IMPRIMIR (MOSTRAR) DATOS

```
print("¡Hoy es un gran día!")
```

IMPRIMIR MÁS DE UN VALOR

```
h=8
print("Debemos descansar", h, "horas")
```

LEER Y GUARDAR UN STRING QUE INGRESA EL USUARIO

```
nombre=input("Tu nombre: ")
```

LEER Y GUARDAR UN NÚMERO QUE INGRESA EL USUARIO

```
edad=int(input("Tu edad: "))
```

DECISIONES

SIMPLES (DOS POSIBILIDADES)

```
n=int(input("Adivina el número: "))
if n==9:
    print("Ganaste un premio")
else:
    print("No adivinaste")
```

MÚLTIPLES (MÁS DE DOS POSIBILIDADES)

```
if n==9:
    print("Ganaste un premio")
elif n<9:
    print("Tu número es menor")
else:
    print("Tu número es mayor")
```

👉 else es siempre opcional | puede haber varios elif 👈

BUCLÉS

FIJOS CON NÚMEROS (IMPRIMIR NÚMEROS ENTRE 10 Y 99)

```
for n in range(10,100):
    print(n)
```

FIJOS CON SECUENCIA (IMPRIMIR CARACTERES DE STRING)

```
for caracter in "1, 2, 3, ¡allá vamos!":
    print(caracter)
```

CONDICIONALES

```
nombre=input("Nombre: ")
while nombre!="Luis":
    print("Esta persona no es Luis")
    nombre=input("Nombre: ")
```

CONTENEDORES

LISTAS

```
nums=[1,2,3]
nums[0]
nums.append(4)
del nums(0)
```

DICCIONARIOS

```
meses={"ene":1, "feb":2}
meses["ene"]
meses["mar"]=3
del meses["feb"]
```

👉 obtener
👉 agregar
👉 borrar

General Information

```
Whitespace matters! Indent where needed.
Import modules with "import modulename"
# This is a comment
print("Hello, World!") # prints to screen
```

Conditional Statements

```
if isSunny:
    print('It's sunny!')
elif 90 <= temp < 100 and bath > 80:
    print('Bath is hot and full!')
elif not ((job == 'qa') or (usr == 'adm')):
    print('Match if not qa or adm')
else:
    print('No match. Job is ' + job)
```

Lists

```
scores = ['A', 'C', 90, 75, 'C']
scores[0] # 'A'
scores[1:3] # 'C', 90
scores[2:] # 90, 75, 'C'
scores[:1] # 'A'
scores[:-1] # 'A', 'C', 90, 75
len(scores) # 5
scores.count('C') # 2
scores.sort() # 75, 90, 'A', 'C', 'C'
scores.remove('A') # removes 'A'
scores.append(100) # Adds 100 to list
scores.pop() # removes the last item
scores.pop(2) # removes the third item
75 in scores # True
```

For Loops

```
grades = ['A', 'C', 'B', 'F']
for grade in grades: # iterate over all vals
    print(grade)

for k,v in enumerate(grades): # using key value pair
    if v=='F':
        grades[k]='A' # change all Fs to As

inv = {'apples': 7, 'peaches': 4}
for fruit, count in inv.items(): # using dictionaries
    print("We have {} {} ".format(count, fruit))
```

```
for i in range(10): # 0 to 9 counting by 1s
for i in range(5, 10): # 5 to 9 counting by 1s
for i in range(9, 2, -1): # 9 to 3 decreasing by 1s
```

Functions

```
def sumNums(numOne, numTwo = 0):
    return numOne + numTwo

print(sumNums(3,4)) # 7
print(sumNums(3)) # 3
```

Numbers

```
total = 3 * 3 # 9
total = 5 + 2 * 3 # 11
cost = 1.50 + 3.75 # 5.25
total = int("9") + 1 # 10
```

Strings

```
title = 'Us and them'
# most list operations work on strings
title[0] # 'U'
len(title) # 11
title.split(' ') # ['Us', 'and', 'them']
''.join(['A', 'B', 'C']) # 'A:B:C'
nine = str(9) # convert int to string
title.replace('them', 'us') # Us and us
```

Tuples

```
Like lists, except they cannot be changed
tuple1 = (1,2,3,"a","z") # Creates tuple
tuple1[3] # 'a'
```

Dictionaries

```
votes = {'red': 3, 'blue': 5}
votes.keys() # ['blue', 'red']
votes['gold'] = 4 # add a key/val
del votes['gold'] # deletes key
votes['blue'] = 6 # change value
len(votes) # 2
votes.values() # [6, 3]
'green' in votes # False
votes.has_key('red') # True
```

While Loops

```
i = 0
while True:
    i += 1
    if i == 3:
        continue # go to next loop
    if i == 7:
        break # end loop
    print(i) # 1 2 4 5 6
```

Class

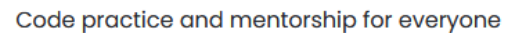
```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def birthYear(self):
        return year - self.age

user = Person('Jimmi', 27)
user.name = 'Jim'
print(user.name) # prints Jim
print(user.birthYear())
```


Por favor toma tu tiempo y pon atención...





OR

EMAIL

Enter your email address

PASSWORD

Enter your password

[Forgot your password?](#)

[Log In](#)

Don't have an account yet? [Sign up for free.](#)

Didn't receive your confirmation email? [Resend it.](#)



Exercism is free for all people, everywhere

Level up your programming skills with **7790 exercises across 78 languages**, and insightful discussion with our dedicated team of welcoming mentors.

Exercism is 100% free forever.



+48 more

Bibliografía

- Solano Soto, J. Introducción a la programación Python. Editorial Tecnológica de Costa Rica.
- Presentación:
 - Ing. Saúl Calderón. MSc.
 - William Mata